PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 15

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum

1. Membuat Register View

- Modifikasi view/auth/register/index.tsx

```
import Link from "next/link";
import style from "./register.module.scss";

const TampilanRegister = () => {
    return (
        <div className={style.register}>
            <h1 className={style.register__title}>Halaman Register</h1>
            <Link href="/auth/login">Ke Halaman Login</Link>
        </div>
    );
};

export default TampilanRegister;
```

- Modifikasi pages/auth/register.tsx

```
import TampilanRegister from "../../view/auth/register";

const HalamanRegister = () => {
    return (
        <TampilanRegister />
    );
};

export default HalamanRegister;
```

- Modifikasi view/auth/register/register.module.scss

```
.register {
    padding: 40px;
}

.register__title {
    font-size: 24px;
    font-weight: bold;
}
```

- Modifikasi view/auth/register/index.tsx

```
import Link from "next/link";
import style from "./register.module.scss";

const TampilanRegister = () => {
    return (
        <div className={style.register}>
            <h1 className={style.register__title}>Halaman Register</h1>
            <div className={style.register__form}>
                <form action="">
                <div className={style.register__form_item}>
                    <label
                    htmlFor="email"
                    className={style.register__form_item_label}
                    >
                    Email
                    </label>
                    <input
                    type="email"
                    id="email"
                    name="email"
                    placeholder="Email"
                    className={style.register__form_item_input}
                    />
                </div>

                <div className={style.register__form_item}>
                    <label
                    htmlFor="Fullname"
                    className={style.register__form_item_label}
                    >
                    Fullname
                    </label>
                    <input
                    type="text"
                    id="Fullname"
                    name="Fullname"
                    placeholder="Fullname"
                    className={style.register__form_item_input}
                    />
                </div>

                <div className={style.register__form_item}>
                    <label
                    htmlFor="Password"
                    className={style.register__form_item_label}
                    >
                    Password
                    </label>
                    <input
                    type="password"
                    id="Password"
                    name="Password"
                    placeholder="Password"
                    className={style.register__form_item_input}
                    />
                </div>

                <button type="submit" className={style.register__form_item_button}>
                    Register
                </button>
                </form>
                <br />
                <p className={style.register__form_item_text}>
                Sudah punya akun? <Link href="/auth/login">Ke Halaman Login</Link>
                </p>
            </div>
        </div>
    );
};

export default TampilanRegister;
```

- Modifikasi view/auth/register/register.module.scss

```scss
.register {
  min-height: 100vh;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  background: linear-gradient(135deg, #1e3c72, #2a5298);
  padding: 20px;

  &__title {
    width: 100%;
    text-align: center;
    font-size: 36px;
    font-weight: 700;
    color: #ffffff;
    margin-bottom: 40px;
  }

  &__form {
    background: #ffffff;
    width: 100%;
    max-width: 420px;
    padding: 40px;
    border-radius: 16px;
    box-shadow: 0 20px 40px rgba(0, 0, 0, 0.15);

    &__item {
      display: flex;
      flex-direction: column;
      margin-bottom: 20px;

      &__label {
        font-size: 14px;
        font-weight: 600;
        margin-bottom: 8px;
        color: #333;
      }

      &__input {
        padding: 12px 14px;
        border-radius: 10px;
        border: 1px solid #ddd;
        font-size: 14px;
        transition: all 0.3s ease;

        &:focus {
          outline: none;
          border-color: #2a5298;
          box-shadow: 0 0 3px rgba(42, 82, 152, 0.2);
        }
      }
    }
```

```scss
    button {
      width: 100%;
      padding: 12px;
      border: none;
      border-radius: 10px;
      background: #2a5298;
      color: #ffffff;
      font-weight: 600;
      cursor: pointer;
      transition: 0.3s ease;

      &:hover {
        background: #1e3c72;
        transform: translateY(-2px);
      }
    }
  }
}

/* Animation */
@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(15px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

/* Responsive */
@media (max-width: 768px) {
  .register {
    &__form {
      padding: 30px 20px;
    }

    &__title {
      font-size: 26px;
    }
  }
}
```

2. Membuat API Register

- Modifikasi servicefirebase.ts

```
my-app > src > lib > db > TS servicefirebase.ts > signUp
```

```ts
import {
  getFirestore,
  collection,
  getDocs,
  getDoc,
  Firestore,
  doc,
  query,
  where,
  addDoc,
} from "firebase/firestore";
import app from "./firebase";
import { stat } from "node:fs";

const db = getFirestore(app);

export async function retrieveProducts(collectionName: string) {
    const snapshot = await getDocs(collection(db, collectionName));
    const data = snapshot.docs.map((doc) => ({
        id: doc.id,
         ...doc.data(),
    }));
    return data;
}

export async function retrieveDataById(collectionName: string, id: string) {
    const snapshot = await getDoc(doc(db, collectionName, id));
    const data = snapshot.data();
    return data;
}

export async function signUp(
    userData: {
        email: string;
        password: string;
        fullname: string;
    },
    callback : Function,
) {
    const q =query(
        collection(db, "users"),
        where("email", "==", userData.email)
    );
    const querySnapshot = await getDocs(q);
    const data = querySnapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));

    if (data.length > 0) {
        callback({
            status: "success",
            message: "User registered successfully",
        });
    }else {
        callback({
            status: "error",
            message: "User already exists",
        });
    }
}
```

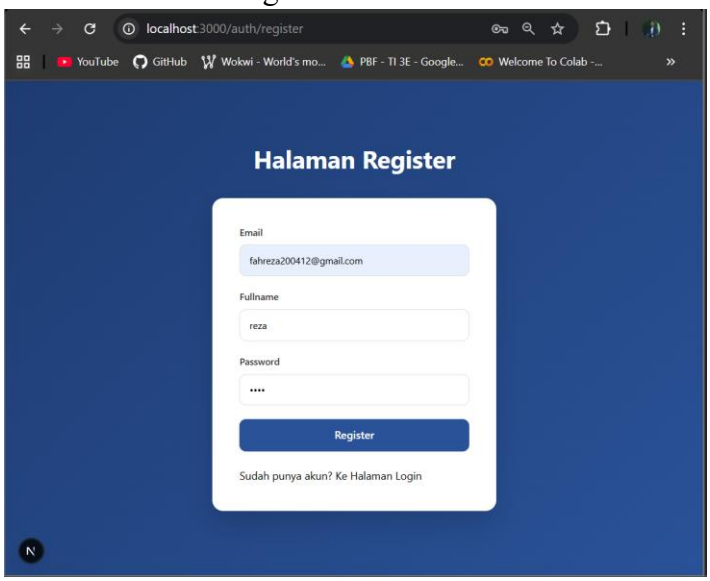
- Buat register.ts pada folder api dan modifikasi

```
import { signUp  } from "@/utils/db/servicefirebase";
import type { NextApiRequest, NextApiResponse } from "next";

type Data = {
    name: string;
    alamat: string;
}

export default async function handler(
    req: NextApiRequest,
    res: NextApiResponse<Data>
) {
    if (req.method === "POST") {
        await signUp(req.body, (result: { status: string; message: string }) => {
            if (result.status === "success") {
                res.status(200).json({ name: result.message, alamat: "" });
            } else {
                res.status(400).json({ name: result.message, alamat: "" });
            }
        });
    } else {
        res.status(405).json({ name: "Method Not Allowed", alamat: "" });
    }
}
```

- Modifikasi view/auth/register/index.tsx

```
import React, { useState } from "react";
import Link from "next/link";
import { useRouter } from "next/router";
import style from "./register.module.scss";

const TampilanRegister = () => {
    const [isLoading, setIsLoading] = useState(false);
    const [error, setError] = useState("");
    const { push } = useRouter();
    const handleSubmit = async (event: React.FormEvent<HTMLFormElement>,) => {
        event.preventDefault();
        setError("");

        const form = event.currentTarget;
        const formData = new FormData(event.currentTarget);
        const email = formData.get("email") as string;
        const fullname = formData.get("Fullname") as string;
        const password = formData.get("Password") as string;

        if (!email) {
            setError("Email wajib diisi");
            return;
        }

        if (!password || password.length < 6) {
            setError("Password minimal 6 karakter");
            return;
        }

        setIsLoading(true);
        const response = await fetch("/api/register", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({ email, fullname, password }),
        });
        // const result = await response.json();
        // console.log(result);
        if (response.status === 200) {
            form.reset();
            setIsLoading(false);
            push("/auth/login");
        } else {
            setIsLoading(false);
            setError(
                response.status === 400
                    ? "Email already exists" : "An error occurred",
            );
        }
    };

    return (
        <div className={style.register}>
            <h1 className={style.register__title}>Halaman Register</h1>
            <div className={style.register__form}>
                <form onSubmit={handleSubmit}>
                <div className={style.register__form_item}>
```

- Akses URL /registrasi



3. Install bcrypt

- npm install bcrypt –force

```
PS E:\PBF_2026\JS_15\my-app> npm install bcrypt --force
npm warn using --force Recommended protections disabled.

added 3 packages, and audited 489 packages in 8s
```

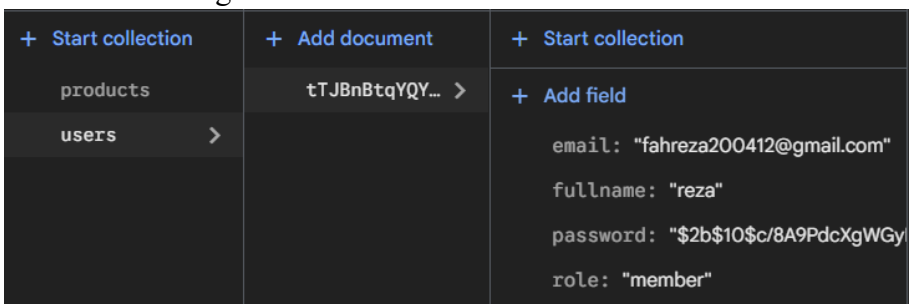
- npm install --save-dev @types/bcrypt –force

```
PS E:\PBF_2026\JS_15\my-app> npm install --save-dev @types/bcrypt -force
npm warn using --force Recommended protections disabled.

added 1 package, and audited 490 packages in 3s
```

- Modifikasi servicefirebase.ts

```
my-app > src > utils > db > servicefirebase.ts > signUp > catch() callback
    doc,
    query,
    where,
    addDoc,
  } from "firebase/firestore";
  import app from "./firebase";
  import { stat } from "node:fs";
  import bcrypt from "bcrypt";
  import { use } from "react";

  const db = getFirestore(app);

> export async function retrieveProducts(collectionName: string) {...
  }

> export async function retrieveDataById(collectionName: string, id: string) {...
  }

  export async function signUp(
      userData: {
          email: string;
          password: string;
          fullname: string;
          role?: string;
      },
      callback : Function,
  ) {
      const q =query(
          collection(db, "users"),
          where("email", "==", userData.email)
      );
      const querySnapshot = await getDocs(q);
      const data = querySnapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));

      if (data.length > 0) {
          callback({
              status: "error",
              message: "User already exists",
          });
      }else {
          userData.password = await bcrypt.hash(userData.password, 10);
          userData.role = "member";
          await addDoc(collection(db, "users"), userData)
              .then(() => {
                  callback({
                  status: "success",
                  message: "User registered successfully",
                  });
              })
              .catch((error) => {
                  callback({
                  status: "error",
                  message: error.message,
                  });
              });
      }
  }
```

- Jalankan registrasi dan cek firestore



- Modifikasi view/auth/register/index/tsx

```
return (
    <div className={style.register}>
        {error && <p className={style.register__error}>{error}</p>}
        <h1 className={style.register__title}>Halaman Register</h1>
```

```
    <button
        type="submit"
        className={style.register__form__item__button}
        disabled={isLoading}
    >
        {isLoading ? "Loading..." : "Register"}
    </button>
```

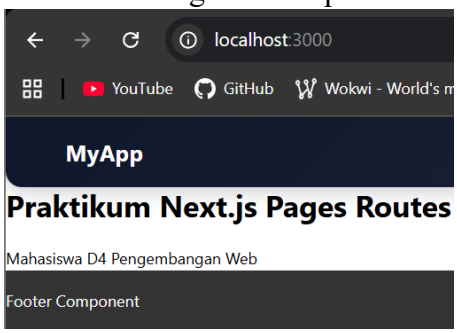
```
} else {
    setIsLoading(false);
    setError(
        response.status === 400
            ? "Email already exists" : "An error occurred",
    );
}
```

- Modifikasi register.module.scss

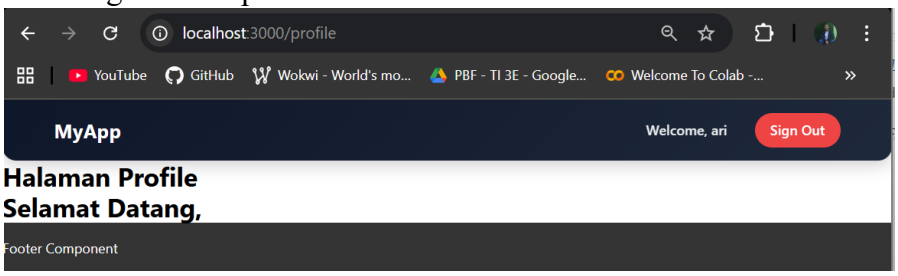
```
&__error {
    color: #e74c3c;
    font-size: 14px;
    margin-top: 8px;
}
```

4. Pengujian

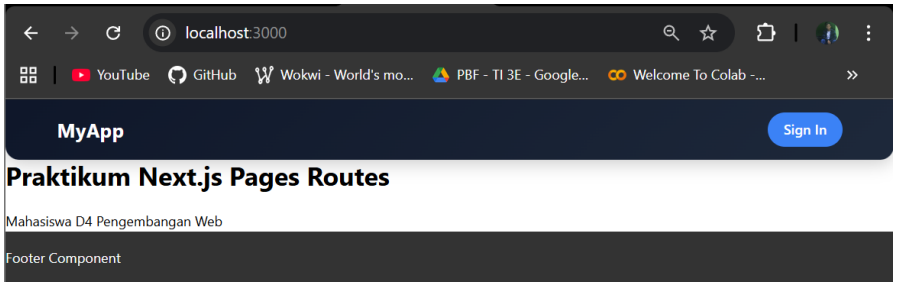
- Belum login akses /profile



- Login akses /profile



- SingOut akses/profile



Tugas Praktikum

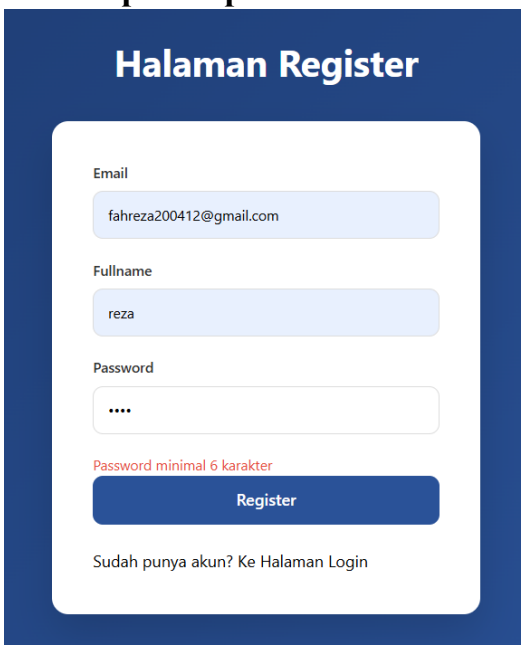
1. **Implementasikan register terhubung database.**
2. **Tambahkan validasi:**
   - Email wajib
   - Password minimal 6 karakter

```
if (!email) {
    setError("Email wajib diisi");
    return;
}

if (!password || password.length < 6) {
    setError("Password minimal 6 karakter");
    return;
}
```

3. **Tambahkan role default "member".**

```
userData.role = "member";
await addDoc(collection(db, "users"), userData)
```

4. **Tampilkan pesan error di UI.**



Pertanyaan Refleksi

1. **Mengapa password harus di-hash?**

Password tidak boleh disimpan dalam bentuk plaintext untuk menghindari risiko kebocoran data dan karena keamanan yang rendah jika database berhasil ditembus. Menggunakan hashing (seperti bcrypt) adalah standar praktik terbaik (best practice) untuk menjaga kerahasiaan data pengguna

2. Apa perbedaan addDoc dan setDoc?

- addDoc: Digunakan untuk membuat dokumen baru dengan ID otomatis yang dihasilkan oleh Firestore.
- setDoc: Digunakan untuk menyimpan dokumen dengan ID spesifik yang ditentukan sendiri oleh pengembang

3. Mengapa perlu validasi method POST?

Validasi ini diperlukan untuk memastikan bahwa API Route hanya memproses pengiriman data yang sah. Jika tidak divalidasi, metode lain (seperti GET) bisa mengakses endpoint tersebut, yang berisiko menyebabkan kesalahan sistem atau celah keamanan (seperti munculnya pesan Method Not Allowed)

4. Apa risiko jika email tidak dicek unik?

Tanpa pengecekan keunikan, satu email dapat digunakan untuk membuat banyak akun. Hal ini menyebabkan redundansi data, kebingungan saat proses login, dan potensi penyalahgunaan akun oleh pihak yang tidak bertanggung jawab

5. Apa fungsi role pada user?

Role berfungsi untuk menerapkan Role-Based Access Control (RBAC). Tujuannya adalah untuk membatasi akses sehingga halaman tertentu (seperti halaman admin) hanya bisa diakses oleh pengguna dengan hak akses yang sesuai, sementara pengguna biasa tidak bisa masuk